A bit of theory as far as relevant to the implementation of `class LagrangeMesh`

The necessary theory - for non-reduced axes - is found in Ryssens et al, PHYSICAL REVIEW C 92, 064318 (2015) *section III.B Lagrange-mesh representation*.

Non-reduced axes

(As from Ryssens et al, PHYSICAL REVIEW C 92, 064318 (2015) *section III.B Lagrange-mesh representation*)

Grid points

Using M evenly space grid point at distance dx, the boundaries of the box are $[-\frac{M}{2}dx, \frac{M}{2}dx]$. The width of the box is $L = Mdx$. We require $M = 2N$ to be even such that there are $N = \frac{M}{2}$ grid points on each side of the origin and the origis is avoided as a grid point.

For non-reduced axes (where the shift parameter σ is required be zero) the grid points are:
[eq 1]

$$\pm\frac{1}{2}dx, \pm\frac{3}{2}dx, \ldots, \pm\frac{2N-3}{2}dx, \pm\frac{2N-1}{2}dx$$

or
[eq 2]

$$x_{\pm i} = \pm(\frac{1}{2} + i)dx = \pm(\frac{2i+1}{2})dx$$

$$i = 0..N-1$$

in order:
[eq 3]

$$-\frac{2N-1}{2}dx, -\frac{2N-3}{2}dx, \ldots, -\frac{3}{2}dx, -\frac{1}{2}dx, \frac{1}{2}dx, \frac{3}{2}dx, \ldots, \frac{2N-3}{2}dx, \frac{2N-1}{2}dx$$

or
[eq 4]

$$x_i = -\frac{2N-1-2i}{2}dx = (\frac{1}{2} + i - N)dx, i = 0..(2N-1)$$

or
[eq 4.1]

$$x_i = -\frac{2N-1-2i}{2}dx = (\frac{1}{2} + i - N)dx, i = -(N-1)..(N-1)$$

Basis functions

The basis functions are plane waves:
[eq 5]

$$\phi_k(x) = \frac{1}{\sqrt{L}}\exp(\frac{2\pi\mathrm{i}}{L}kx)$$

where i is the imaginary unit (not the index i), and

$$k = \pm\frac{1}{2}, \pm\frac{3}{2}, \ldots, \pm\frac{2N-3}{2}, \pm\frac{2N-1}{2}$$

or, since kdx is a grid point, say, the i-th, x_i:
[eq 5.1]

$$\phi_{x_i}(x) = \frac{1}{\sqrt{L}}\exp(\frac{2\pi\mathrm{i}}{L}\frac{x_i}{dx}x) = \frac{1}{\sqrt{L}}\exp(\frac{2\pi\mathrm{i}}{Ldx}x_i x)$$

When x is a grid point, e.g. $x = x_j$, we have

$$\phi_{x_i}(x_j) = \frac{1}{\sqrt{L}}\exp(\frac{2\pi\mathrm{i}}{Ldx}x_i x_j) = \frac{1}{\sqrt{L}}\exp(\frac{2\pi\mathrm{i}}{2Ndxdx}(\frac{1}{2}+i-N)dx(\frac{1}{2}+j-N)dx))$$

$$= \frac{1}{\sqrt{L}}\exp(\frac{\pi\mathrm{i}}{N}(\frac{1}{2}+i-N)(\frac{1}{2}+j-N))$$

$$= \frac{1}{\sqrt{L}}\exp(\frac{\pi\mathrm{i}}{N}(\frac{1}{4}+\frac{1}{2}j-\frac{1}{2}N+i\frac{1}{2}+ij-iN-N\frac{1}{2}-Nj+N^2))$$

$$= \frac{1}{\sqrt{L}}\exp(\frac{\pi\mathrm{i}}{N}(\frac{1}{4}+\frac{1}{2}(i+j)-N+ij-(i+j)N+N^2))$$

(this isn't very helpful.)

Interpolation

The Lagrange interpolation functions are:
[eq 6]

$$f_i(x) = \frac{1}{2N}\frac{\sin(\frac{\pi}{dx}(x-x_i dx))}{\sin(\frac{\pi}{dx}\frac{x-x_i dx}{2N})}$$

By construction, the Lagrange interpolation functions $f_i(x)$. have the property of being equal to 1 at the i-th mesh point, x_i, and 0 at all others:
[eq 7]

$$f_i(x_j) = \delta_{ij}$$

Note that when evaluating $f_i(x_i)$, both the numerator and the denominator are 0, and Python yields a `nan`. We need to invoke l'Hôpitals's rule to obtain the result.
[eq 7.1]

$$\lim_{x\to x_i}\frac{1}{2N}\frac{\sin(\frac{\pi}{dx}(x-x_i dx))}{\sin(\frac{\pi}{dx}\frac{x-x_i dx}{2N})}$$

$$=\frac{1}{2N}\lim_{x\to x_i}\frac{\sin'(\frac{\pi}{dx}(x-x_i dx))}{\sin'(\frac{\pi}{dx}\frac{x-x_i dx}{2N})}$$

$$=\frac{1}{2N}\lim_{x\to x_i}\frac{\frac{\pi}{dx}\cos(\frac{\pi}{dx}(x-x_i dx))}{\frac{\pi}{2Ndx}\cos(\frac{\pi}{dx}\frac{x-x_i dx}{2N})}$$

$$=\frac{2N}{2N}\lim_{x\to x_i}\frac{cos(0)}{cos(0)}=1$$

In order to avoid testing for $\frac{0}{0}$ it suffices to add $\varepsilon\approx 1\mathrm{e}-9$ to x_i to avoid the `nan` and yield something close to 1.

The arguments of the two sine functions in eq 6 are the same, apart from a factor $1/2N$:
[eq 8]

$$f_i(x)=\frac{1}{2N}\frac{\sin(A_i(x)))}{\sin(\frac{A_i(x)}{2N})}$$

$$A_i(x)=\frac{\pi}{dx}(x-x_i dx)=\pi(\frac{x}{dx}-x_i)$$

This can be exploited for efficiency in a computation.
An arbitrary function $h(x)$ taking the values $h_i=h(x_i)$ on the grid points can be interpolated on an arbitrary point $x\in[-Ndx,Ndx]$ as:
[eq 9]

$$h(x)=\sum_{i=0}^{2N-1}h(x_i)f_i(x)$$

This is essentially a dot product $\mathbf{f}\cdot\mathbf{h}$.

Derivatives

The 1st and 2nd order derivatives of the Lagrange interpolation functions are:
[eq 10]

$$D^{(1)}_{ji}=\frac{df_i(x)}{dx}\bigg|_{x=x_j}=\begin{cases}(-1)^{i-j}\frac{\pi}{2Ndx}\frac{1}{sin(\pi(i-j)/2N)}, & \text{for } i\neq j.\\ 0, & \text{for } i=j.\end{cases}$$

$$D^{(2)}_{ji}=\frac{d^2f_i(x)}{dx^2}\bigg|_{x=x_j}=\begin{cases}(-1)^{i-j+1}2(\frac{\pi}{2Ndx})^2\frac{cos(\pi(i-j)/2N)}{sin^2(\pi(i-j)/2N)}, & \text{for } i\neq j.\\ -\frac{\pi^2}{3dx^2}(1-\frac{1}{(2N)^2}), & \text{for } i=j.\end{cases}$$

By applying these to the expansion $\phi(x)=\sum_{i=0}^{2N-1}\phi(x_i)f_i(x)$ we obtain
[eq 11]

$$\frac{\mathbf{dh}}{\mathbf{dx}}=\frac{dh(x)}{dx}\bigg|_{x=x_j}=\sum_{i=0}^{2N-1}\frac{df_i(x)}{dx}\bigg|_{x=x_j}h(x_i)=\sum_{i=0}^{2N-1}D^{(1)}_{ji}h(x_i)=\mathbf{D}^{(1)}\mathbf{h}$$

So, the column vector $\frac{\mathbf{dh}}{\mathbf{dx}}$ of the derivatives of $\mathbf{h}$ at all grid points is found as a matrix product of $\mathbf{D^{(1)}}$ with the column vector of the values of $\mathbf{h}$ at all the gridpoints. As the 2nd derivative forms a matrix as well, we also have
[eq 12]

$$\frac{\mathbf{d^2h}}{\mathbf{dx^2}} == \mathbf{D}^{(2)}\mathbf{h}$$

Reduced axes

Grid points

On reduced axes only the N grid points to the right of the origin are kept:
[eq 13]

$$\frac{1}{2}dx, \frac{3}{2}dx, \ldots, \frac{2N-3}{2}dx, \frac{2N-1}{2}dx$$

or
[eq 14]

$$x_i = (i + \frac{1}{2})dx$$

$$i = 0..N-1$$

(In de case of reduced axis a shift $\sigma < dx$ may be subtracted from each grid point.)

Interpolation

An arbitrary function $h(x)$, $x \in [0, Ndx]$ taking the values $h_i = h(x_i)$ on the grid points on a reduced axis can be interpolated as :
[eq 15]

$$h(x) = \sum_{i=0}^{2N-1} h_i[f_i(x) \pm f_{-i}(x)]$$

where $+$, resp. $-$, is selected if $h(x)$ is symmetric, resp. antisymmetric, and $f_{-i}(x)$ is the Lagrange interpolation function corresponding to the i-th grid point to the left of the origin:
[eq 16]

$$f_{-i}(x) = \frac{1}{2N}\frac{\sin(\frac{\pi}{dx}(x - x_{-i}dx))}{\sin(\frac{\pi}{dx}\frac{x-x_{-i}dx}{2N})} = \frac{1}{2N}\frac{\sin(\frac{\pi}{dx}(x + (\frac{1}{2}+i)dx))}{\sin(\frac{\pi}{dx}\frac{x+(\frac{1}{2}+i)dx}{2N})}$$

This time we have a sum or a difference of two dot products $\mathbf{h} \cdot \mathbf{f}_+ \pm \mathbf{h} \cdot \mathbf{f}_-$
Again the the arguments of the two sine functions are the same, apart from a factor $1/2N$.

General remark on interpolation with Lagrange functions

On a 1D grid with 6 grid points at $-1.25, -.75, -.25, .25, .75, 1.25$ on the interval $[-1.5, 1.5]$, these are the 6 Lagrange functions:

“MOCCaPy/tests/mocca/mesh/lagrange_function_0.png” could not be found.

“MOCCaPy/tests/mocca/mesh/lagrange_function_1.png” could not be found.

“MOCCaPy/tests/mocca/mesh/lagrange_function_2.png” could not be found.

“MOCCaPy/tests/mocca/mesh/lagrange_function_3.png” could not be found.

“MOCCaPy/tests/mocca/mesh/lagrange_function_4.png” could not be found.

“MOCCaPy/tests/mocca/mesh/lagrange_function_5.png” could not be found.

“MOCCaPy/tests/mocca/mesh/lagrange_functions.png” could not be found.

Clearly, each one yields 1 at one grid point and 0 at the others. Note also that they are antiperiodic: they reenter the box at the opposite edge, but **with a sign change**. Consequently, a constant function cannot be interpolated because it is periodic. The sum of the 6 Lagrange functions is shown below. it is definitely not the constant function $f(x) = 1$.

“MOCCaPy/tests/mocca/mesh/sum_lagrange_functions.png” could not be found.

According to the discussion in github issue 52 periodic functions can be interpolated with Lagrange functions provided they vanish at the boundary of the interval.

Derivatives

The formula for the derivative of a function h expanded on a reduced grid is found easily by extending the column vector $\mathbf{h}$ (of length N) on the reduced grid as
[eq 17]

$$\begin{bmatrix} \pm\mathbf{g} \\ -- \\ \mathbf{h} \end{bmatrix}$$

with $\mathbf{g}$ containing the same elements as $\mathbf{h}$ but in reverse order, if the function h is symmetric, and its negatives in reverse order, if it antisymmetric as if they were mirrored by a horizontal line between the two, and then forming the matrix product
[eq 18]

$$\mathbf{D}\begin{bmatrix} \pm\mathbf{g} \\ \mathbf{h} \end{bmatrix} = \begin{bmatrix} \mathbf{D}^- \\ \mathbf{D}^+ \end{bmatrix}\begin{bmatrix} \pm\mathbf{g} \\ \mathbf{h} \end{bmatrix} = \begin{bmatrix} \mathbf{D}^-\begin{bmatrix} \pm\mathbf{g} \\ \mathbf{h} \end{bmatrix} \\ ---- \\ \mathbf{D}^+\begin{bmatrix} \pm\mathbf{g} \\ \mathbf{h} \end{bmatrix} \end{bmatrix}$$

The upper half of $\mathbf{D}$, $\mathbf{D}^-$, produces the derivatives on the negative x-axis, and the lower half of $\mathbf{D}$, $\mathbf{D}^+$, produces the derivatives on the positive x-axis. The upper part and the lower part are obviously related by symmetry: if the function $h(x)$ is symmetric, then the derivative is antisymmetric and *vice versa*. Thus we are only interested in the lower part. Writing $\mathbf{D}^+$ as $[\mathbf{D}_{ll}|\mathbf{D}_{lr}]$ (ll stands for lower-left and lr for lower-right quadrant of $\mathbf{D}$) we find:
[eq 19]

$$\mathbf{D}^+ \begin{bmatrix} \pm\mathbf{g} \\ \mathbf{h} \end{bmatrix} = [\mathbf{D}_{ll}|\mathbf{D}_{lr}] \begin{bmatrix} \pm\mathbf{g} \\ \mathbf{h} \end{bmatrix} = [\mathbf{E}_{ll}|\mathbf{D}_{lr}] \begin{bmatrix} \pm\mathbf{h} \\ \mathbf{h} \end{bmatrix}$$

where $\mathbf{E}_{ll}$ is obtained from $\mathbf{D}_{ll}$ by reversing the order of the rows. Reversing the order of the rows of $\mathbf{D}_{ll}$ forces us to also reverse the order of the elements of $\pm\mathbf{g}$, which then becomes, by definition $\pm\mathbf{h}$. Finally, we obtain:
[eq 20]

$$\pm\mathbf{E}_{ll}\mathbf{h} + \mathbf{D}_{lr}\mathbf{h} = [\mathbf{D}_{lr} \pm \mathbf{E}_{ll}]\mathbf{h}$$

or
[eq 21]

$$\frac{\mathbf{dh}}{\mathbf{dx}} = [\mathbf{D}_{lr}^{(1)} \pm \mathbf{E}_{ll}^{(1)}]\mathbf{h}$$

where we have reintroduced the superscript $^{(1)}$ to indicate the first order derivative. By the same reasoning, we obtain for the 2nd derivative:
[eq 22]

$$\frac{\mathbf{d}^2\mathbf{h}}{\mathbf{dx}^2} = [\mathbf{D}_{lr}^{(2)} \pm \mathbf{E}_{ll}^{(2)}]\mathbf{h}$$

N-dimensional grids

The full Cartesian 3D representation of a function $h(\mathbf{r})$, where $\mathbf{r} = [x \quad y \quad z]$ (3D case), is then provided (for the 3D case) by
[eq 23]

$$h(\mathbf{r}) = \sum_{ijk} h_{ijk} f_i(x) f_j(y) f_k(z)$$

where the number of discretization points does not have to be the same in each direction.

Note that f_i, f_j and f_k are generally different objects, even if accidentally the indices i, j and k are identical, as they pertain, resp., to the x-axis, the y-axis and the z-axis, and the grid spacing is not necessarily the same.

Derivatives

In this case, the derivative matrices $\mathbf{D}^{(1)}$ and $\mathbf{D}^{(2)}$ have to be set up separately for each direction, taking into account wether the axis is reduced or not.

Basis functions

The basis functions are plane wave products of the different axes:
[eq 24]

$$\Phi_{klm}(x,y,z) = \phi_k(x)\phi_l(y)\phi_m(z) = \frac{1}{\sqrt{L_x}}\frac{1}{\sqrt{L_y}}\frac{1}{\sqrt{L_z}}\exp(\frac{2\pi\mathrm{i}}{L_x}kx)\exp(\frac{2\pi\mathrm{i}}{L_y}ly)\exp(\frac{2\pi\mathrm{i}}{L_z}mz)$$

$$= \frac{1}{\sqrt{L_xL_yL_z}}\exp(2\pi\mathrm{i}(\frac{kx}{L_x}+\frac{ly}{L_y}+\frac{mz}{L_z})$$

$$k = \pm\frac{1}{2}, \pm\frac{3}{2}, \ldots, \pm\frac{N_x-1}{2}$$

$$l = \pm\frac{1}{2}, \pm\frac{3}{2}, \ldots, \pm\frac{N_y-1}{2}$$

$$m = \pm\frac{1}{2}, \pm\frac{3}{2}, \ldots, \pm\frac{N_z-1}{2}$$

$$= \frac{1}{\sqrt{L_xL_yL_z}}\exp(2\pi\mathrm{i}(\mathbf{k}\cdot\mathbf{r})$$

$$\mathbf{k} = \begin{bmatrix}\frac{k}{L_x} & \frac{l}{L_y} & \frac{m}{L_z}\end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix}x & y & z\end{bmatrix}$$

Note that ϕ_k, ϕ_l and ϕ_m are generally different objects, even if accidentally the indices k, l and m are identical, as they pertain, resp., to the x-axis, the y-axis and the z-axis.
Alternatively, in the spirit of eq 5.1:
[eq 24.1]

$$\mathbf{k} = \begin{bmatrix}\frac{x_i}{L_xdx} & \frac{y_j}{L_ydy} & \frac{z_k}{L_zdz}\end{bmatrix}$$

where the x_i, y_j, z_k are grid coordinates in the x, y and z directions.

Interpolation

As described by eq 23 Interpolating a scalar quantity h at a single point requires a sum over all grid points which may be costly (speaking of working interactively). If h needs to be interpolated on a large number of points, p,

$$\mathbf{r} = \begin{bmatrix}x_0 & y_0 & z_0\\ x_1 & y_1 & z_1\\ x_2 & y_2 & z_2\\ \vdots & \vdots & \vdots\\ x_{p-1} & y_{p-1} & z_{p-1}\end{bmatrix}$$

it will be advantageous to move the loop over the points $0..p-1$ inside the loop over ijk product $h_{ijk}f_i(x)f_j(y)f_k(z)$.

In case h is not a scalar quantity but a tensor it is advantageous to move the loop over the components between the loop over the p interpolation points and the loop over ijk:

```
for all grid points ijk:
    for all components h of H
        for all interpolation points 0..p
            accumulate h_ijk * f_i(x_p) * f_j(y_p) * f_k(z_p) over ijk
```

The inner loop can be evaluated as a function over a numpy array:

```
for all grid points ijk:
    for all components h of H
        h(r) +=  h_ijk * f_i(r[:,0]) * f_j(r[:,1]) * f_k(r[:,2])
```

Furthermore, in case e.g. the x axis is reduced, according to eq 15 one must replace $f_i(x)$ by $(f_i(x) \pm f_{-i}(x))$, where the sign is $+$ if f is symmetric and $-$ if f is antisymmetric.